Table of Content

1 Initialization / Main menu....4
2 Utilities....5
2.1 Persistence....5
2.1.1 Introduction....5
2.1.2 Limitations / Known issues....5
2.1.3 Files....6
2.1.4 Populating persistence_datarefs.cfg....7
2.1.5 Persistence menu....7
2.1.6 Configuration options in *preferences.cfg*....9
2.2 Engine Damage....9
2.2.1 Introduction....9
2.2.2 Design concepts....10
2.2.3 Engine profile....13
2.2.4 Engine damage menu....15
2.2.5 Usage....16
2.2.6 Configuration options....16
2.3 Oxygen system....17
2.3.1 Introduction....17
2.3.2 Features....17
2.3.3 Known issues....17
2.3.4 Oxygen System menu....17
2.3.5 Configuration options....18
2.4 Noise-cancelling headset....19
2.4.1 Introduction....19
2.4.2 Features....19
2.4.3 Known issues....20
2.4.4 Headset menu....20
2.4.5 Configuration options....21

2.5 Miscellaneous small utilities....22
2.5.1 Introduction....22
2.5.2 Miscellaneous menu....22
2.5.3 General configuration options....23
2.5.4 Power monitor....23
2.6 Unmaintained utilities....25
3 XLuaUtils developer reference....25
3.1 Limitations....25
3.2 Global variables and paths....26
3.3 Printing, logging and debugging....26
3.3.1 Basics....26
3.3.2 Debug menu....26
3.3.3 Functions....27
3.4 Configuration tables....27
3.4.1 Basics....28
3.4.2 Functions....28
3.5 Preferences....29
3.5.1 Basics....29
3.5.2 Functions....29
3.6 Menus....30
3.7 The XLuaUtils window....30
3.7.1 Basics....31
3.7.2 Window Menu....31
3.7.3 Functions....32
3.8 Notifications....33
3.8.1 Basics....33
3.8.2 Functions....34
3.9 Dataref Handlers....35
3.9.1 Dataref Tables....35
3.9.2 Reading/Writing from/to Datarefs....37

1 Initialization / Main menu

A working XLuaUtils installation is indicated by the presence of an "XLuaUtils" submenu in the aircraft menu (composed of the aircraft's name) in X-Plane's main menu bar (Image 1).

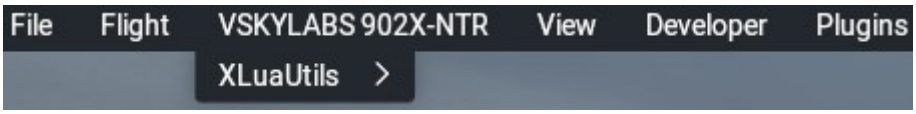


Image 1: XLuaUtils main menu

Without a *preferences.cfg* file in the "xluautils" folder, the main menu looks as in Image 2. For the debug menu, see chapter 3.3.



Image 2: XLuaUtils main menu before initialization

Menu Item	Functionality
Initialize XLuaUtils	Generates a *preferences.cfg* file in the "xluatils" folder containing the initial values of any submodule making use of XLuaUtils' preferences system (see chapter 3.5)

Once a *preferences.cfg* has been created or was detected at startup, XLuaUtils is fully initialized and the entire menu tree, including that of any utility, is available (Image 3).

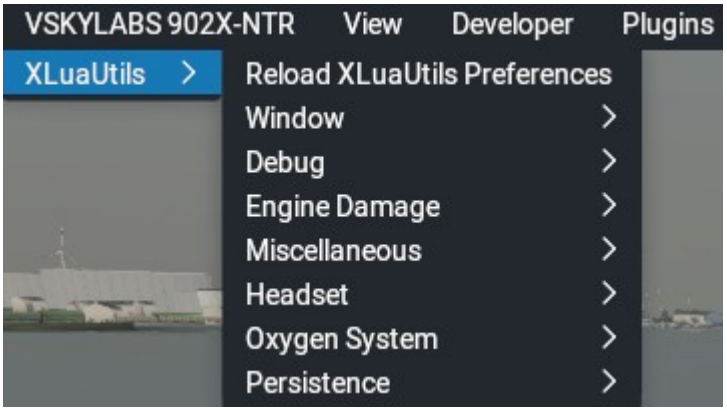


Image 3: XLuaUtils main menu after initialization

Menu Item	Functionality
Initialize XLuaUtils	Reads *preferences.cfg* from the "xluautils" folder. Use this function to reload preferences values that have been altered via manual edit of the file.

2 Utilities

XLuaUtils ships with a variety of (more or less?) useful utilities that facilitate aircraft handling or add some other useful functions not found in default X-Plane 12.

2.1 Persistence

2.1.1 Introduction

XLuaUtils' persistence utility can be used to save the user aircraft's state.

It parses a list of datarefs at startup, whose values are either manually or automatically written to a persistence state file. This state file is then parsed at X-Plane session initialization and the values are written back to these datarefs. The module only works for the aircraft running XLuaUtils (i.e. the currently active user aircraft).

2.1.2 Limitations / Known issues

To get this out of the way first, the persistence system comes with the following limitations and known issues:

- Datarefs that are missing from the aircraft's scripts or because of internal X-Plane code changes are discarded during persistence module initialization, they will not be saved later on.
- Not all datarefs trigger appropriate system responses from the aircraft. This is largely due to the way that a developer has coded the systems. A good rule of thumb is that the more default X-Plane datarefs an aircraft has, the greater is the chance for persistence working reliably.
- True in-flight saving is difficult and requires tracking quite few physics datarefs. See *xluautils/Config Files/Persistence/zz_Deep_Initialization.cfg* for a working example dor the default X-Plane Cessna 172SP.
- Custom datarefs created by third party aircraft must be writable in order to be used by the persistence module. If a third-party aircraft uses XLua or SASL to drive its systems, it could be that its custom datarefs are not initialized with a handler which would make them writable (if a custom dataref can be edited with DataRefTool, it is writable).

There is a procedure to make **all** custom xlua variables automatically writable (not possible for SASL driven aircraft):
1. Make a backup of plugins/xlua/init.lua.
2. In xlua/init.lua, find the code block starting with "function create_dataref(name,type,notifier)".
3. A few lines below, there will be this line: "dref = XluaCreateDataRef(name,type,"no",nil)".
4. Replace the "no" with "yes", i.e: "dref = XluaCreateDataRef(name,type,"yes",nil)".
5. Save and close init.lua.

- Despite all the care taken in finding the correct datarefs and putting them into a sensible order, it may be that some third party aircraft simply do not react too well toward third party tools trying to write to their datarefs.
  In general, the simpler the add-on, the higher the chance of success for a 100% correct initialization.
- FMS and GPS flight plans and configurations are not supported unless they are stored within datarefs.
- There is no way to restore dataref values from the start of the X-Plane session prior to persistence file reading, so the only solution to a messed up persistence save file is to delete *xluautils/persistence_save.txt*, disable persistence auto-loading from the menu and restart X-Plane or change to another aircraft and back and start over.

2.1.3 Files

These persistence files are stored in the *xluautils* folder:

- XLuaUtils' *preferences.cfg* stores the configuration for the persistence utility,
- *persistence_datarefs.cfg* contains the list of datarefs that are to be monitored
- *persistence_save.txt* is the save file containing the dataref values

An empty _"Persistence_datarefs.cfg"_ is generated at XLuaUtils initialization (see chapter [5.1](#5.1) if none is present.

2.1.4 Populating persistence_datarefs.cfg

Persistence_datarefs.cfg is a regular text file and may be read and edited with any text editor like Notepad (apps like MS Word are not recommended though). The header section of the file contains information on how to find the relevant datarefs used by the current aircraft.

There are (at least) two techniques to find the relevant datarefs used by the aircraft:

- Digging through the aircraft's *[...]cockpit.obj* file (or whichever file contains "ATTR_manip…" information) by searching for a switch's tool tip or "ATTR_manip…" items.
  Note that: " ATTR_manip" items that contain some form of the term "command" do not contain dataref information. Commands are generally not supported by the persistence utility. Find the equivalent dataref and see if it's writable.
- Monitor datarefs that recently changed by user input with DataRefTool

Datarefs used by manipulators are guaranteed to be writable, so dig for those first.

If a default datarefs has to be used to compensate for a "command" type manipulator, cross-check that it is writable by consulting *Resources/plugins/datarefs.txt*.

Custom datarefs created by third party aircraft are supported, as long as they are writable (see note in "Limitations" above).

Invalid datarefs that can not be found in X-Plane are discarded during the initial parsing of *persistence_datarefs.cfg* and will not be used, so there is minimal risk of crashing X-Plane.

Only the name of the dataref is required, not the index, i.e. *my_example_dataref[12]* would be added as *my_example_dataref* to *persistence_datarefs.cfg.*

There is no limit to the number of datarefs in *persistence_datarefs.cfg*.

Comments are denoted with a hash sign ("#") at the beginning of a line.

Dataref order might matter if the aircraft's systems logic dictates one.

Example *persistence_datarefs.cfg* files for some add-on aircraft can be found in *xluautils/Config Files/Persistence*. These may be used as a starting point or template. Contributions are welcome.

2.1.5 Persistence menu

The "Persistence" sub-menu (Image 4) is available when a *persistence_datarefs.cfg* file was found during XLuaUtils' initialization.

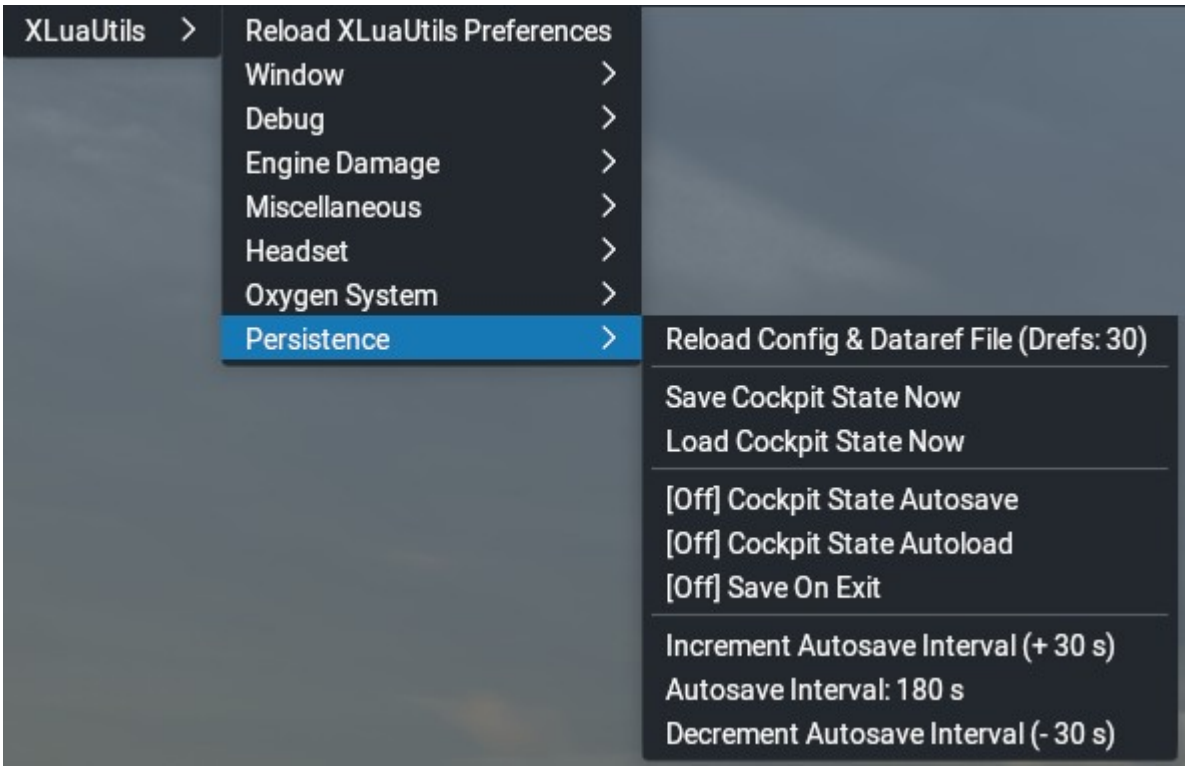


Image 4: XLuaUtils persistence menu

Menu Item	Functionality
Reload Config & Dataref File (Drefs: n)	Reads persistence module related data from _"preferences.cfg"_, parses _"persistence_datarefs.cfg"_ and then reads the current values of all tracked datarefs. The menu entry will display the amount of currently tracked datarefs.
Save Cockpit State Now	Saves the values of the tracked dataref to *xluautils/persistence_save.txt*
Load Cockpit State Now	Loads the values of the tracked datarefs' values from *xluautils/persistence_save.txt*
[On/Off] Cockpit State Autosave	Toggles the timed autosave feature on or off. The first autosave is performed after a certain delay to avoid conflicts with other XLuaUtils file access functions. After the first autosave, a timer is started that will save the persistence state in a specific interval.
[On/Off] Cockpit State Autoload	Toggles the autoload feature on or off. Autoloading of persistence data from *persistence_save.txt* is started during X-Plane's "flight_start" event during aircraft initialization. To avoid conflicts with custom systems logic from other scripts or plugins, autoload is performed after a certain delay, which may be configured in *preferences.cfg* (see below).
[On/Off] Save On Exit	Triggers a persistence save operation during X-Plane's "aircraft_unload" event, e.g. when shutting down X-Plane or changing the aircraft.
Increment Autosave Interval (+ n s)	Increments the autosave interval by a certain amount of time. The increment may be adjusted in preferences.cfg (see below).
Autosave Interval: n s	Only displays the current autosave interval and does not constitute an interactive element.
Decrement Autosave Interval (- n s)	Decrements the autosave interval by a certain amount of time. The decrement may be adjusted in preferences.cfg (see below).

2.1.6 Configuration in preferences.cfg

The lines in XLuaUtils' *preferences.cfg* relevant to the persistence utility are prefixed with "PERSISTENCE".

```
PERSISTENCE,Autoload:string,0:number
PERSISTENCE,AutoloadDelay:string,5:number
PERSISTENCE,Autosave:string,0:number
PERSISTENCE,AutosaveInterval:string,120:number
PERSISTENCE,AutosaveIntervalDelta:string,30:number
PERSISTENCE,AutosaveDelay:string,10:number
PERSISTENCE,SaveOnExit:string,0:number
```

Parameter	Value range	Description
Autoload	0 or 1 (default: 0)	Enable or disable the autoloading of persistence data.
AutoloadDelay	> 0	Delay, in seconds, before autoloading of the persistence data commences
Autosave	0 or 1 (default: 0)	Enable or disable the autosaving of persistence data.
AutosaveInterval	>0 (default:120)	The interval, in seconds, in which persistence data is autosaved.
AutosaveIntervalDelta	>0 (default: 30)	Increment/decrement for the autosave interval adjustment, in seconds.
AutosaveDelay	>0 (default: 10)	Delay, in seconds, before the first autosave
SaveOnExit	0 or 1 (default: 0)	Enable or disable the saving of persistence data on X-Plane session exit or aircraft unload.

When altering these parameters, only adjust the numbers and nothing else.

Changes to *preferences.cfg* may be applied immediately with the "Reload Config & Dataref File" function from the "Persistence" menu or the "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.2 Engine Damage

2.2.1 Introduction

The engine damage utility will add consequences to exceeding operational limitations. These limitations depend on engine type (e.g. torque and CHT for props, ITT and EGT for jets, etc.) and are either derived from the aircraft's ACF file or, in case that no sensible limits were set by the original aircraft author, by user-specific thresholds set in *engine_profile.cfg* (see below).

Each supported parameter may be enabled individually to adapt "Engine Damage" to a given engine type.

Engine damage accumulation is dynamic, determined by the amount of redline exceedance and some randomization.

Engine failure due to stress is probabilistic, with the chance being greater the more stress an engine accumulates.

2.2.2 Design concepts

2.2.2.1 Supported parameters

The supported engine parameters for "Engine Damage" are:

- cylinder head temperature (CHT)
- exhaust gas temperature (EGT)
- interstage turbine temperature (ITT)
- manifold pressure (MP)
- first spool (or prop) rotation speed (N1)
- second spool (or engine) rotation speed (N2)
- oil pressure (OIL_P)
- oil temperature (OIL_T)
- propeller RPM (RPM_P)
- Torque (TRQ)

Any of these parameters may be enabled at any time. To avoid premature or excesssive engine damage, there should be a sensible limit set in the *.acf* file (and detected from a dataref) or in *engine_profile.cfg*.

2.2.2.2 Limitations / Known issues

For mixed engine type aircraft, e.g. piston and jet engines, parameters peculiar to each engine type will have to be enabled at the same time. "Engine Damage" will evaluate each of these parameters for each engine. Correct handling of redline values for each engine type mounted on the aircraft is therefore exclusively up to X-Plane.

I may also have overdone the randomization, but I like randomization too much as things would get boring otherwise.

2.2.2.3 Base values

The red line limit for a supported parameter, supplied by the dataref or override value, serves as a baseline with a given normal durability, in seconds. For example, the engine stress at a redline value for N1 at 100% can be tolerated for 300 seconds (or 5 minutes) before failure. This value pair (engine parameter and associated time) is used as the lower limit by "Engine Damage".

As this would make things too predictable and not dynamic enough, a second value pair is used in the form of a scalar for the redline value and a second time parameter, which, for example, scales a 100% N1 redline by 1.25 to obtain 125 % N1. At this N1, the engine would last, say, 10 seconds. This engine parameter - time value pair is used as the upper limit by "Engine Damage"
Engine damage interpolates between these two value pairs to obtain the time the engine lasts at the current value. To stick with the example, a N1 of 110 % would yield around 200 seconds of time to failure.

2.2.2.4 Randomization

To make things more unpredictable and avoid simultaneous engine failure in multi engine aircraft, the upper and lower limits (engine parameter and stress tolerance time) are randomized between 98 % and 104 % (defaults) of their nominal value during each aircraft (re)load.

The slope between the randomized values for the linear interpolation is calculated and stored per flight session to save a few CPU cycles.

2.2.2.5 Stress recession

Since it's easier to add stress to an engine than remove it, there is a separate parameter to scale the current stress rate when engine stress is receding. This parameter is set in the engine profile (see below) and randomized each flight session between 98 % and 102 % of the nominal value.

2.2.2.6 Engine monitoring

"Engine Damage" will loop through all available engines on the aircraft and, if the particular engine has not failed, will monitor said engine for all enabled parameters.

The currently monitored parameter is checked against its nominal base limit (as specified by the dataref or engine profile) and, if an exceedance is recorded, engine stress will be accumulated at a rate determined by the (previously randomized) slope between the (previously randomized) lower and upper limits.

A notification will be displayed for a few seconds or, if pinned, as long as the engine is accumulating stress.

If the given engine parameter falls below the redline parameter, engine stress will decrease by the randomized stress recess rate. Therefore, there is a chance that a subsequent redline exceedance will start with the previously accumulated stress.

Accumulated engine stress can not exceed the maximum stress tolerance limit based on the (previously randomized) upper limit and can not be lower than zero.

2.2.2.7 Engine failure

When an engine has accumulated stress greater than zero, there is a chance of failure. At the lower engine limit, the default chance is 0.0001, i.e. 1 in 10000 per second (which is the default script refresh rate), while at the upper limit, the default chance is 1 in 100 per second. The failure chance is linearly interpolated between these limits.

The failure system uses a "lucky number" between 1 and 10, which is assigned to each monitored parameter of each engine during module initialization at each aircraft (re)load. Each script update cycle, this "lucky number" is compared against a randomly picked number from a pool whose size is determined by the current engine stress level.

When the "lucky number" matches the randomly picked number, the engine will fail.

The failure mode depends on the engine parameter that caused the failure. Supported modes are:

- Engine fire (CHT, EGT, ITT, oil temperature)
- Loss of power (N1, N2)
- Oil pump failure (oil pressure)
- Propeller detachment (propeller RPM)
- Shaft breakage (torque)

On engine failure, a notification will be displayed for 60 seconds (default) or optionally as long as the failure state persists.

2.2.2.8 Engine repair

All engine damage may be repaired at any time from the menu to avoid frustration. Be your own game master!

2.2.3 Engine profile

2.2.3.1 Location

The engine profile information is stored in
[Aircraft folder]/plugins/xlua/scripts/xluautils/engine_profile.cfg.

This file can be edited with any text editor and is generated when "Engine Damage" is initialized. Its file header contains information regarding line structuring and more.

Example engine profiles for some aircraft can be found in *xluautils/Config Files/Engine Damage*. These may be used as a starting point or template. Contributions are welcome.

2.2.3.2 File specification

"DMG_" lines contain data for a given supported engine parameter (see section "Supported Parameters"). All of these lines share the same amount and order of values, e.g.:
`DMG_CHT,0,-1,°C,1.5,300,10,0.75`

Value index	Value (Range)	Description
1	DMG_xxx	"DMG_" plus the identifier of the engine parameter (e.g. "DMG_EGT")
2	1 or 0	The given parameter is monitored
3	-1 or > 0	-1: Use dataref/_.acf_ parameter limit, any other value: Use the specified limit. Value must be greater than zero.
4	[See table below]	The unit used for the manual limit (value 3). See the following table for supported units.
5	> 1.0	Scalar for the parameter limit at index 3 to obtain the upper limit (see section "Base values").
6	> 0	Time (in seconds) for which the engine can take stress when this engine parameter is at the lower limit specified at index 3.
7	> 0 and < [time at upper limit]	Time (in seconds) for which the engine can take stress when this engine parameter is at the upper limit calculated from index 3 and 4.
8	> 0	The current engine stress accumulation rate is scaled by this value when engine stress is decreasing.

The units stated in the following table may be used when setting manual limits for "DMG_" properties in *engine_damage.cfg.*

Parameter(s)	Unit(s)
CHT, EGT, ITT, OIL_T	°C, °F
MP	inHg
N1, N2	%

Parameter(s)	Unit(s)
OIL_P	psi
RPM_P	1/min
TRQ	%, Nm, lb-ft

If no manual limit is specified, the unit will be determined by the dataref. Mis-typed or unsupported units will be ignored.

"Randomize" lines control engine parameter randomization, e.g.:
`RandomizeLimit,0.98,1.04`

Value index	Value (Range)	Description
1	"Randomize[Limit/Damage]"	Line identifier. "Limit" randomizes the engine parameter's limits, while "Damage" (at the moment) randomizes an engine parameter's stress recession rate
2	< 1.0	Lower end of the Limit/Damage randomization range
3	> 1.0	Upper end of the Limit/Damage randomization range

The "FailureChance" line controls the failure chance for this aircraft's engines:

`FailureChance,0.001,0.1`

Value index	Value (Range)	Description
1	"FailureChance"	Line identifier
2	< 1.0	Failure chance at the lower limit of the engine stress range. Divide 1 by this value to get your failure chance (1 in n) per second. 1.0 provides a virtually guaranteed failure
3	> 1.0	Failure chance at the upper limit of the engine stress range. Divide 1 by this value to get your failure chance (1 in n) per second. 1.0 provides a virtually guaranteed failure

2.2.4 Engine damage menu

The "Engine Damage" sub-menu is available when XLuaUtils' *persistence.cfg* file was found during initialization.

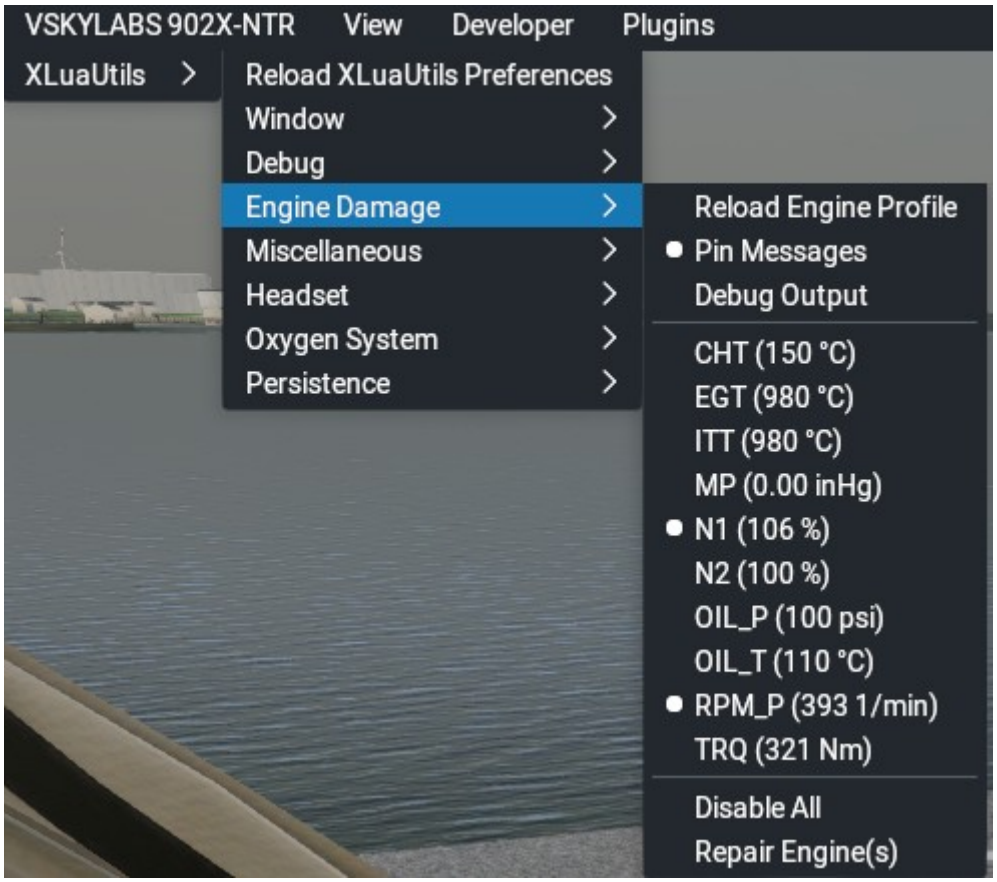


Image 5: XLuaUtils engine damage menu

Menu Item	Functionality
[Generate/Reload] Engine Profile	Click to write or read *engine_profile.cfg*.
Pin Messages	When active, engine failure notifications will remain on screen instead of disappearing after a set time (see "Configuration" below)
CHT ... TRQ	Shows the available properties that can be monitored and their limit and unit as determined from the datarefs or *engine_profile.cfg*. Click to toggle the monitoring of this parameter. **Caution:** Will rewrite *engine_profile.cfg*!
Disable All	Shortcut for turning off the monitoring of all enabled parameters.
Repair Engine(s)	Clears all engine failures. **Note that all failed engines will have to be restarted again!**

2.2.5 Usage

Generate an engine profile file with "Generate Engine Profile" from the menu.

Warning: An existing *engine_profile.cfg* will be overwritten, so back up your old one before clicking this!

If you’re happy with all default values, simply enable the parameters you wish to have monitored from the menu **or** override limits and enable parameters within *engine_profile.cfg* and then use "Reload Engine Profile".
Warning: Do not use the "CHT" ... "TRQ" or "Disable All" menu items before finishing your edits and reloading *engine_profile.cfg* or your customized file will be overwritten!

Test fly your aircraft. I'm 99 % sure you will try to test the engines' limits now.

2.2.6 Configuration in preferences.cfg

"Engine Damage" may be configured in XLuaUtils’ preferences.cfg. Relevant lines are prefixed with ENGINEDAMAGE:

```
ENGINEDAMAGE,MainTimerInterval:string,1:number
ENGINEDAMAGE,Notify_Pin:string,1:number
ENGINEDAMAGE,Notify_Time:string,30:number
```

Parameter	Value range	Description
MainTimerInterval	> 0	Sets the refresh interval, in seconds, for Engine Damage's main timer. Should be greater than zero. **Warning:** The lower the interval, the more frequent the failure chance calculation and thus the higher the chance of engine failure!
Notify_Pin	1 or 0	Same as the menu's "Pin Messages" item
Notify_Time	> 0 (default: 30)	Sets how long a notification will be displayed if "Pin Messages" is *not* enabled

When altering these parameters, only adjust the numbers and nothing else.

Changes to *preferences.cfg* may be applied immediately with the "Reload Engine Profile" function from the "Engine Damage" menu or the "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.3 Oxygen system

2.3.1 Introduction

This utility provides a supplemental oxygen system (or overrides a present oxygen system) without having to edit an aircraft's PlaneMaker model. It also has an automation mode and provides warnings about pilot hypoxia.

2.3.2 Features

The Oxygen System utility is basically a user interface for various default X-Plane datarefs and provides some degree of automation as well as popup notifications that will inform the pilot about the onset (above 14,000 ft felt altitude) of and impairment by (above 15,000 ft felt altitude) hypoxia. When the system is enabled, the pilot will be informed when the fill level in the bottle(s) is down to 75%, 50%, 25% and 0%.
With the oxygen system enabled (mask on), the oxygen flow will be automatically increased when the pilot is above 10000 ft felt altitude and will be decreased when the pilot is below 8000 ft felt altitude.

2.3.3 Known issues

On first use, the oxygen bottle will be filled to just 50 litres. In case the aircraft has a bigger bottle installed, simply click “Refill Bottle" from the menu (see below) before going on your first flight. After that, the oxygen volume in the bottle will be tracked persistently.

2.3.4 Oxygen System menu

The "Oxygen System" sub-menu is available when XLuaUtils’ *persistence.cfg* file was found during initialization.

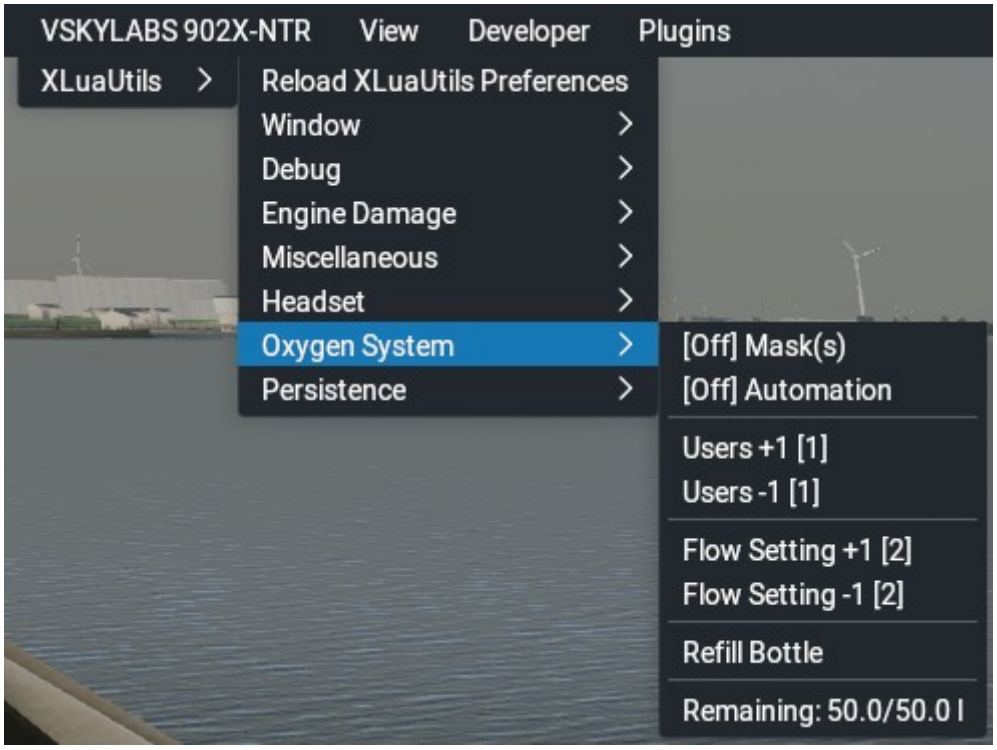


Image 6: XLuaUtils oxygen system menu

Menu Item	Functionality
[On/Off] Masks	Immediately puts on the oxygen mask. When set to off, it will also turn off mask automation.
[On/Off] Automation	When enabled, will automatically put on the oxygen mask above 12500 ft.
Users +1 [n]	Increments the number of oxygen users by 1. Should be adjusted to the number of people on board that require oxygen (probably all). The number in square brackets is the current number of users.
Users -1 [n]	Decrements the number of oxygen users by 1. The number in square brackets is the current number of users .
Flow Setting +1 [n]	Increments the oxygen flow setting by one, decreasing the effects of hypoxia more quickly at the cost of increased oxygen consumption. The number in square brackets is the current flow setting.
Flow Setting -1 [n]	Decrements the oxygen flow setting by one. The number in square brackets is the current flow setting.
Refill Bottle"/"[Can Not Refill]	When on the ground, click here to refill the oxygen bottle to full capacity.
Remaining: n/m l	Just for information: The remaining and total capacity of the oxygen bottle on board, in litres.

2.3.5 Configuration in preferences.cfg

The oxygen system may be configured in XLuaUtils' preferences.cfg. Relevant lines are prefixed with OXYGENSYSTEM.

```
OXYGENSYSTEM,Automation:string,0:number
OXYGENSYSTEM,MasksOn:string,0:number
OXYGENSYSTEM,MainTimerInterval:string,1:number
OXYGENSYSTEM,BottleCapacityLiters:string,50:number
OXYGENSYSTEM,BottleRemainingLiters:string,50:number
OXYGENSYSTEM,FlowSetting:string,2:number
OXYGENSYSTEM,Users:string,1:number
OXYGENSYSTEM,PilotAltitude:string,10000:number,8000:number
```

Parameter	Value range	Description
Automation	0 or 1 (default: 0)	Enable or disable the automatic donning of oxygen masks.
MasksOn	0 or 1 (default: 0)	Enable or disable the oxygen masks.
MainTimerInterval	>0 (default: 1)	Timer interval, in seconds, for oxygen mask automation checks.
BottleCapacityLiters	>0 (default: 50)	The capacity of the oxygen bottle, in litres, if not defined in PlaneMaker.
BottleRemainingLiters	>0 (default: 50)	The remaining amount of oxygen, in litres, in the bottle
FlowSetting	0 to 8; (default: 2)	The flow setting for the oxygen mask(s).
Users	>0 (default:1)	The number of users connected to the oxygen system.
PilotAltitude	>0 (default: 10000, 8000)	The felt altitudes, in feet, at which the oxygen flow will be automatically increased (first value) and decreased (second value).

When altering these parameters, only adjust the numbers and nothing else.
Changes to *preferences.cfg* can be applied immediately with "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.4 Noise-cancelling headset

2.4.1 Introduction

XLuaUtil's noise-cancelling headset module provides a customizable one-size-fits all solution to loud engines. It decreases engine, environment, exterior, fan, ground, interior, prop, warning and weather volume levels, while retaining all other sound levels. The headset will only affect volume levels when inside a fmod sound space or in an interior view (fallback mode).

2.4.2 Features

The noise-cancelling function can be toggled from the menu or enabled/disabled automatically. When automated, the headset is only active when all engines are running.

Volume levels from the beginning of the X-Plane session are stored during module initialization for later restoration. These volume levels are restored during X-Plane's "aircraft_unload" event, e.g. when changing aircraft or shutting down X-Plane.

2.4.3 Known issues

Reloading the aircraft or its Lua scripts, or an X-Plane program crash with the NC function active will prevent the volume level restoration logic from running. Thus, the volume levels present at script initialization will be considered defaults to which the noise-cancelling will be applied. This leads to much lower than desired sound levels requiring manual restoration from X-Plane's sound menu.

2.4.4 Headset menu

The "Headset" sub-menu is available when XLuaUtils' *persistence.cfg* file was found during initialization.

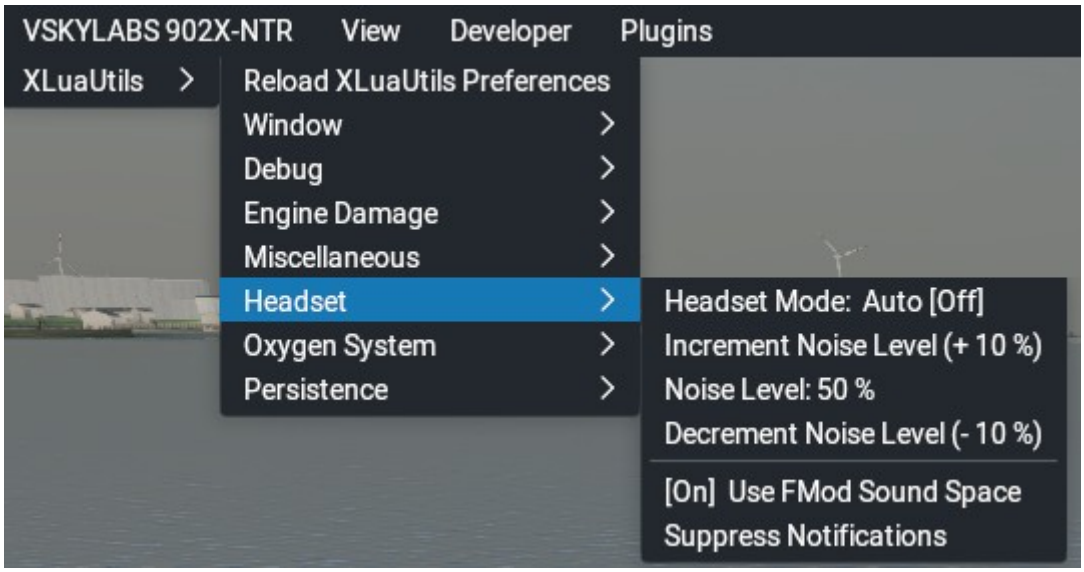


Image 7: XLuaUtils noise-cancelling headset menu

Menu Item	Functionality
Headset Mode [On / Off / Auto [On/Off]]	Cycles through the headset mode. “On” means the headset is always on when inside of a fmod soundspace, “off” means no headset and “Auto” puts the headset on when all aircraft engines are operating, with the current status indicated in brackets.
Increment Noise Level (+ n %)	Increments the headset noise level by a certain percentage (adjustable in *preferences.cfg*, see below).
Noise Level (n %)	Only indicates the current noise reduction level.
Decrement Noise Level (+ n %)	Decrements the headset noise level by a certain percentage (adjustable in *preferences.cfg*, see below).
[On/Off] Use FMod Sound Space	When on, the dataref *sim/operation/sound/inside_any* is used to determine whether the user is inside the aircraft or not. When off, s*im/graphics/view/view_is_external* is used.
Suppress Notifications	Suppresses any notifications the “Headset” utility sends to the XLuaUtils window (see chapter 3.8)

2.4.5 Configuration in preferences.cfg

The noise-cancelling headset may be configured in XLuaUtils’ preferences.cfg. Relevant lines are prefixed with NCHEADSET.

```
NCHEADSET,HeadsetState:string,0:number
NCHEADSET,NoiseCancelLevel:string,0.5:number
NCHEADSET,NoiseCancelLevelDelta:string,0.1:number
NCHEADSET,MainTimerInterval:string,0.5:number
NCHEADSET,FmodCompliant:string,1:number
NCHEADSET,NotifySuppress:string,0:number
```

Parameter	Value range	Description
HeadsetState	0 to 2 (default: 0)	The headset state, where 0 is off, 1 is on and 2 is automatic.
NoiseCancelLevel	0 to 1 (default: 0.5)	The amount (from 0.0 to 1.0) of outside noise level with the headset on.
NoiseCancelLevelDelta	0 to 1 (default: 0.1)	The increment or decrement that the level of noise cancellation can be adjusted by.
MainTimerInterval	>0 (default: 0.5)	The time interval, in seconds, at which the headset's main timer function checks for the inside
FmodCompliant	0 or 1 (default: 1)	Use the Fmod dataref to determine if user is inside the aircraft, otherwise use a different dataref.
NotifySuppress	0 or 1 (default: 0)	Suppresses notification output on headset state changes.

When altering these parameters, only adjust the numbers and nothing else.

Changes to *preferences.cfg* can be applied immediately with "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.5 Miscellaneous small utilities

2.5.1 Introduction

This is a collection of smaller functions collected in a sub-menu that can further make the X-Plane pilot's life easier.

2.5.2 Miscellaneous menu

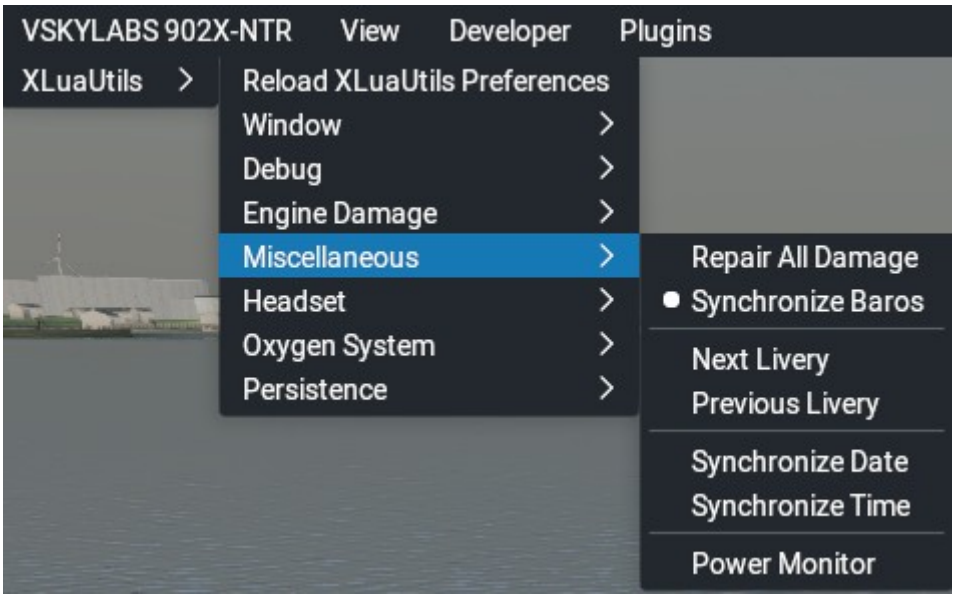


Image 8: XLuaUtils miscellaneous small utilities menu

Menu Item	Functionality
Repair All Damage / “[Can Not Repair]”	This resets the value of all X-Plane's failure datarefs (>500) to zero (from a value of six, indicating a failure). Aircraft must be stationary on the ground with all engines off in order to use this. If the aircraft can not be repaired, the menu entry will read "[Can Not Repair]".
Synchronize Baros	When enabled, synchronizes the pilot's, co-pilot's and standby altimeter when any of them are changed.
Next Livery	Switches to the next aircraft livery.
Previous Livery	Switches to the previous aircraft livery.
Synchronize Date	Synchronizes X-Plane's day and month to the system's day and month. Does not consider the year! **Warning: Might not work properly in X-Plane versions newer than 12.3!**
Synchronize Time	Synchronizes X-Plane's local time to the system's local time. **Warning: Might not work properly in X-Plane versions newer than 12.3!**
Power Monitor	Outputs engine power data to the notification window

2.5.3 General configuration in preferences.cfg

The lines for the miscellaneous utilities in XLuaUtils’ *preferences.cfg* are prefixed with MISC_UTILS.

```
MISC_UTILS,MainTimerInterval:string,1:number
MISC_UTILS,SyncBaros:string,1:number
```

Parameter	Value range	Description
MainTimerInterval	>0 (default: 1)	Sets the refresh interval, in seconds, for the main timer used by misc

Parameter	Value range	Description
		utilities. Should be greater than zero.
SyncBaros	0 or 1 (default: 0)	1 = Barometer synchronization is enabled (see "Menu" above)

When altering these parameters, only adjust the numbers and nothing else.

Changes to *preferences.cfg* can be applied immediately with "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.5.4 Power monitor

The Power Monitor is very much inspired by the one in SimCoders' Reality Expansion Packs and helps monitoring relevant engine parameters without having to enable XP's data outputs or looking at the instruments.

2.5.4.1 Features

Power Monitor offers the following outputs, per engine:

- Engine power as a percentage
- Optional: Scaled engine power (in case it is required)
- Engine horsepower
- Engine RPM
- Engine manifold pressure
- Cylinder head temperature (CHT)
- Fuel flow in various units (see below), but always per hour

2.5.4.2 Limitations

At the moment, Power Monitor is exclusively configured for reciprocating engines.

Guessing the temperature units from the ACF file, while possible, is not something I wish to implement (at the moment), so both units (°C/°F) are displayed at any time, with the user required to figure out which one X-Plane uses (hint open the aircraft in PlaneMaker and then look in the "Standard" --> "Systems" --> "Limits 1/2" menu).

2.5.4.3 Usage

Toggle Power Monitor on/off with the "Power Monitor" menu item (see above).

Power Monitor will check X-Plane's main throttle, mixture and prop lever input datarefs for changes and then display the engine parameters listed above for each running engine in a notification window. If no change from these levers has been determined for 5 seconds or if the engine produces negative power during deceleration, the notification window will close itself again.

However, it is also possible to have it open permanently as long as the engine produces positive power (see below).

2.5.4.4 Configuration in preferences.cfg

Power Monitor may only be configured in XLuaUtils' *preferences.cfg*, with the relevant lines prefixed with "MISC_UTILS":

```
MISC_UTILS,PowerMonitor:string,1:number
MISC_UTILS,PowerMonitorDisplayTime:string,5:number
MISC_UTILS,PowerMonitorScalar:string,1.0:number
MISC_UTILS,PowerMonitorInputChange:string,0.005:number
MISC_UTILS,PowerMonitorFuelUnit:string,kg:string
```

Parameter	Value range	Description
PowerMonitor	0 or 1 (default: 0)	1 = Power Monitor is enabled.
PowerMonitorDisplayTime	<1 or >1 (default: 5)	>1: The Power Monitor notification will stay visible for this amount of time after no throttle, mixture or prop change has been detected. <1: The Power Monitor notification is permanently open as long as the engine produces positive power.
PowerMonitorScalar	<1.0 or >1.0 (default: 1.0)	Enables a scaled output of the engine power percentage when smaller or greater than 1.0. Could be useful in some scenarios.
PowerMonitorInputChange	0 to 1 (default: 0.005)	The delta that the throttle, mixture or power lever must move in order to trigger the Power Monitor notification.
PowerMonitorFuelUnit	Kg lbs gal_avgas gal_jet-a l_avgas l_jet-a	Outputs the fuel flow in the specified format. Fuel flow is always [unit] per hour. Conversion factors: 2.20462 lbs/kg, Avgas: 5.87 lbs/gal (per X-Plane) and 0.719 kg/l, Jet-A: 6.66 lbs/gal and 0.796 kg/l.

When altering these parameters, only adjust the numbers and nothing else.

Changes to *preferences.cfg* can be applied immediately with "Reload XLuaUtils Preferences" from the "XLuaUtils" menu.

2.6 Unmaintained utilities

There are two currently unmaintained utilities in XLuaUtils:

- “Automixture”, providing mixture capabilities akin to that found on a P&W R-1830 type engine.
- “Attach Objects”, which can be used to display objects (such as a GPU or drop tanks) outside the aircraft.

Because of their status as unmaintained, their documentation is only available in the “documentation” folder.

3 XLuaUtils developer reference

This section contains information about some XLuaUtils features useful for XLua script developers.

3.1 Limitations

Because XLua name spaces are completely local, reading variables from other scripts is not possible. If you want to use any of XLuaUtils’ functions in airplane related scripts, add them as a sub-module at the end of the `submodules` section in *xluautils.lua*. Any XLua limitations apply, especially regarding table scope in XLua 1.1 or newer. Any table must be - and is being - treated as local. XLuaUtils’ capabilities do not encompass the full extent of X-Plane’s API (yet?).

3.2 Global variables and paths

XLuaUtils will populate the variables stated in 1 at script startup. All of these variables are available in all of XLuaUtils’ submodules.

Table 1: Global Xlua Utils variables

Variable	Description
ACF_Folder	Complete path of the folder that the .acf file of the user aircraft is located in.
ACF_Filename	Name of the user aircraft’s .acf file.
XLuaUtils_Path	Complete path of the XLuaUtils root folder.
XLuaUtils_PrefsFile	Complete path to the XLuaUtils preferences file, including filename.

3.3 Printing, logging and debugging

See *Submodules/xluautils_core_debugging.lua*.

3.3.1 Basics

XLuaUtils logs to X-Plane's *Log.txt* and to *xluautils/z_XLuaUtils_log.txt*. XLuaUtils' log file is recreated at every plugin start of XLua. All log entries are timestamped.

Debug level logging (with information that is not necessary for day-to-day usage) can be controlled from the *"XluaUtils/Debug"* menu (see Image 9) or in the preferences file.

3.3.2 Debug menu

The debug menu is always available, even without *preferences.cfg* present.

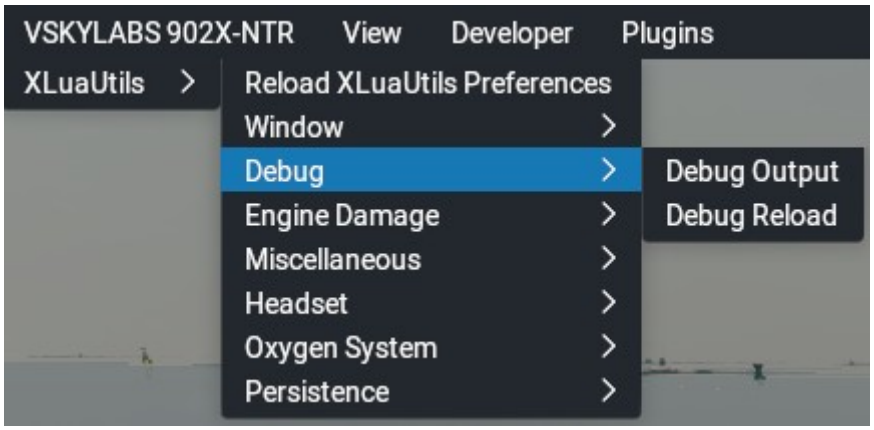


Image 9: Debug menu

Menu Item	Functionality
Debug Output	Toggles the debug output for the logging system on or off.
Debug Reload	Potentially deprected.

3.3.3 Functions

Print to X-Plane's developer console (and stdout) with *PrintToConsole(...)*. The same as using "print(inputstring)".

PrintToConsole(inputstring)		
Parameter	**Value(s)**	**Description**
inputstring	[Any string]	An input string, e.g. "Hello".

Write exclusively to *z_XluaUtils_log.txt* with *WriteToLogFile(...).*

WriteToLogFile(inputstring)		
Parameter	**Value(s)**	**Description**
inputstring	[Any string]	An input string, e.g. “Hello”.

Write to X-Plane’s *Log.txt* and *z_XluaUtils_log.txt* with *LogOutput(...).*

LogOutput(inputstring)		
Parameter	**Value(s)**	**Description**
inputstring	[Any string]	An input string, e.g. “Hello”.

Print a debug-level message to X-Plane’s *“Log.txt”* and *xluautils/z_XluaUtils_log.txt”* with *DebugLogOutput(...).*

DebugLogOutput(inputstring)		
Parameter	**Value(s)**	**Description**
inputstring	[Any string]	An input string, e.g. “Hello”.

3.4 Configuration tables

See *XluaUtils/Submodules/xluautils_core_common.lua*.

3.4.1 Basics

XLuaUtils can store configuration data for itself and any utility implemented as a submodule in a configuration table. This table can then be written to or read from the preferences file (see section 3.5).

Any submodule wishing to use the module configuration and preferences system must use a table configured as per the following example:

```
local MyConfigTable = {

{"EXAMPLE"}, -- A unique identifier string indicating the owner of values
stored in the preferences file.

{"MyParameter",12}, -- A sub-table with a setting parameter. The first value
of this table must always be a unique string identifying the parameter.

{"MyOtherParameter",50,20,"Yes"}, -- Parameter sub-tables support numbers and
strings, but no further sub-tables.

}
```

Note that, as of XLua 1.1 (or later), a configuration table has to be defined as local and is constrained to the XLuaUtils sub-module in which it is declared.

3.4.2 Functions

Read a value from a table with *Table_ValGet(...).*

Table_ValGet(inputtable,target,subtabtarget,index)		
Parameter	**Value(s)**	**Description**
inputtable	[String]	The name of the table that data is stored in.
target	[String]	The identifier string of a sub-table.
subtabtarget	[String] or [Number]	The identifier string or index of a sub-sub-table. May be omitted or set to zero.
index	[Number]	The index of a value in the sub-table. This may be omitted, which will pick the value after the identifier string (index 2).

Using the example table above,

```
Table_ValGet(MyConfigTable,"MyOtherParameter",0,4)
```

would return `"Yes"`.

Write a value to a preferences table with *Table_ValSet(...).*

Table_ValSet(outputtable,target,subtabtarget,index,newvalue)		
Parameter	**Value(s)**	**Description**
inputtable	[String]	The name of the table that data is stored in.
target	[String]	The identifier string of a sub-table.
subtabtarget	[String] or [Number]	The identifier string or index of a sub-sub-table. May be omitted or set to zero.
index	[Number]	The index of a value in the sub-table. This may be omitted, which will pick the value after the identifier string (index 2).
newvalue	[Number] or [String]	The new value of the table item

Using the example table above

```
Table_ValSet(MyConfigTable,"MyOtherParameter",0,4,"No")
```

would set the `"Yes"` at index 4 to `"No"`.

3.5 Preferences

See X*luaUtils/Submodules/xluautils_core_common.lua*.

3.5.1 Basics

Preferences from XLuaUtils and any of its submodules are stored in *preferences.cfg* and saved in the *XLuaUtils* folder by default.

To read and write to the preferences file, the input table must be structured as outlined in section 3.4.

3.5.2 Functions

Read from the preferences file with *Preferences_Read(...).*

Preferences_Read(inputfile,outputtable)		
Parameter	**Value(s)**	**Description**
inputfile	[String]	The path to the target file (usually the *XLuaUtils_PrefsFile* variable).
outputtable	[String]	The name of the table that parsed data is output to. Only lines matching the unique identifier string of *outputtable* are read, the rest is ignored.

Write to the preferences file with *Preferences_Write(...).*

Preferences_Write(inputtable,outputfile)		
Parameter	**Value(s)**	**Description**
inputtable	[String]	The name of the table that data is read from.
outputfile	[String]	The path to the target file, usually the *XluaUtils_PrefsFile* variable.

The writing process is selective, i.e. any data present in *outputfile* that is not part of *inputtable* is retained.

3.6 Menus

See most of the *“util_…”* files in “*XluaUtils/Submodules”* contain examples for menus.

XLuaUtils provides multiple examples for implementing menus via X-Plane API calls through LuaJIT’s foreign function interface (FFI). **Note that menus are very volatile and prone to crash X-Plane when encountering syntax or logic errors!**

The basic concept behind menus in XLuaUtils is the following:

- Menu items are read from an initial table, in which the fist element provides the string for the menu’s name, which is not used in the menu build, watchdog or callback functions.

All other menu items may be stated as strings from table element 2. **Index 2 is the start index for all callback or watchdog entries.**
Separators (horizontal lines) may be defined with a table element containing the `[Separator]` string.

- The main *build* function for the menu creates an entry in a parent menu (either X-Plane's aircraft menu or XLuaUtils' menu). The function then iterates over the table containing the menu entries to populate the menu.
- The menu *callback* function controls the interaction of the menu entries with variables. Use this to interact with variables or datarefs.
- The menu *watchdog* is used to alter menu element strings to reflect value changes of certain variables. It is called during menu building, from the menu callback function or may be called from a timer.

3.7 The XLuaUtils window

See *XluaUtils/Submodules/xluautils_core_window.lua*.

3.7.1 Basics

The XLuaUtils window is a window that displays notifications and other text (Image 10).

In VR, the window will be decorated, while in 2D, it is undecorated (no frame). Even when undecorated, the XLuaUtils window can be moved within the X-Plane window by clicking and dragging it with the left mouse button.

The XLuaUtils window will automatically open when a new notification is displayed and will automatically close when no content is displayed anymore. The minimum notification level that will open the window can be controlled from the menu (see below).

If the number of lines to be displayed exceeds the window height, page control buttons will be available at the bottom of the window. These can then be used to flip through the content. Whenever notifications are active, the window will be locked to page 1, but when no more notifications are displayed, the active page will be reset to the one that the user was previously on.



Image 10: The XLuaUtils window

3.7.2 Window Menu

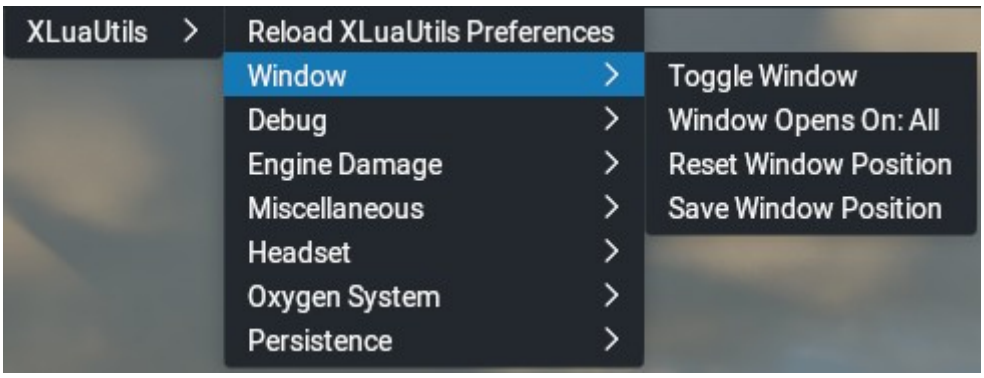


Image 11: The XLuaUtils window menu

Menu Item	Functionality
Toggle Window	Opens the XLuaUtils window. This will display some lines in a “test” group.
Window Opens On: ...	This sets the minimum level of a window message, such as a notification, that can open the XLuaUtils window. Clicking this item cycles through the following levels: “All” – All notifications and messages may open the window. “Caution” - notifications and messages that at least have the level “Caution” may open the window.

Menu Item	Functionality
	“Warning” - notifications and messages that at least have the level “Warning” may open the window.
Reset Window Position	Resets the window position to is default value.
	An optional unique ID for this line. May be nil or "none". Useful for finding and replacing this exact line with *XluaUtils_Window_ReplaceLine*.

3.7.3 Functions

Add a line of text to the window with *XluaUtils_Window_AddText(...).*

XLuaUtils_Window_AddText(group,string,colorkey,id)		
Parameter	**Value(s)**	**Description**
group	[Any string]	A group name for the line used to display a section header, e.g. "Personal Greeting". **Passing an group name is mandatory!**
string	[Any string]	The message to display n the window. If the text string contains new line or return marks (“\n” or “\r”), it will be split into multiple lines on parsing, e.g. “Hi there!\nBeautiful day, isn’t it?”
colorkey	“Nominal”, “Success”, “Caution”, “Warning”	“Nominal” is white, “Success” is green, “Caution” is orange and “Warning” is red. Passing “nil” will default to “Nominal” (i.e. white).
id	[Any string]	An optional unique ID for this line. May be nil or "none". Useful for finding and replacing this exact line with *XluaUtils_Window_ReplaceLine*.

Remove all lines that belong to a particular group with *XluaUtils_Window_RemoveText(...).*

XLuaUtils_Window_RemoveText(group)		
Parameter	**Value(s)**	**Description**
group	[Any string]	The name of the group that is to be removed.

Replace a line by its ID with *XluaUtils_Window_ReplaceLine(...)*. **Use this to update a line in the window from a flight or timer loop.**

XLuaUtils_Window_ReplaceLine(id,string,colorkey)		
Parameter	**Value(s)**	**Description**
id	[Any string]	The unique ID of the item to be replaced.
string	[Any string]	The new string for the item.
colorkey	“Nominal”, “Success”, “Caution”, “Warning”	“Nominal” is white, “Success” is green, “Caution” is orange and “Warning” is red. Passing “nil” will default to “Nominal” (i.e. white).

`XluaUtils_Window_ClearAll()` clears the entire text stack.

3.8 Notifications

See *XluaUtils/Submodules/xluautils_core_window.lua*.

3.8.1 Basics

To provide short-time (or permanent) information, XLuaUtils displays notifications in the XluaUtils window (Image 12).

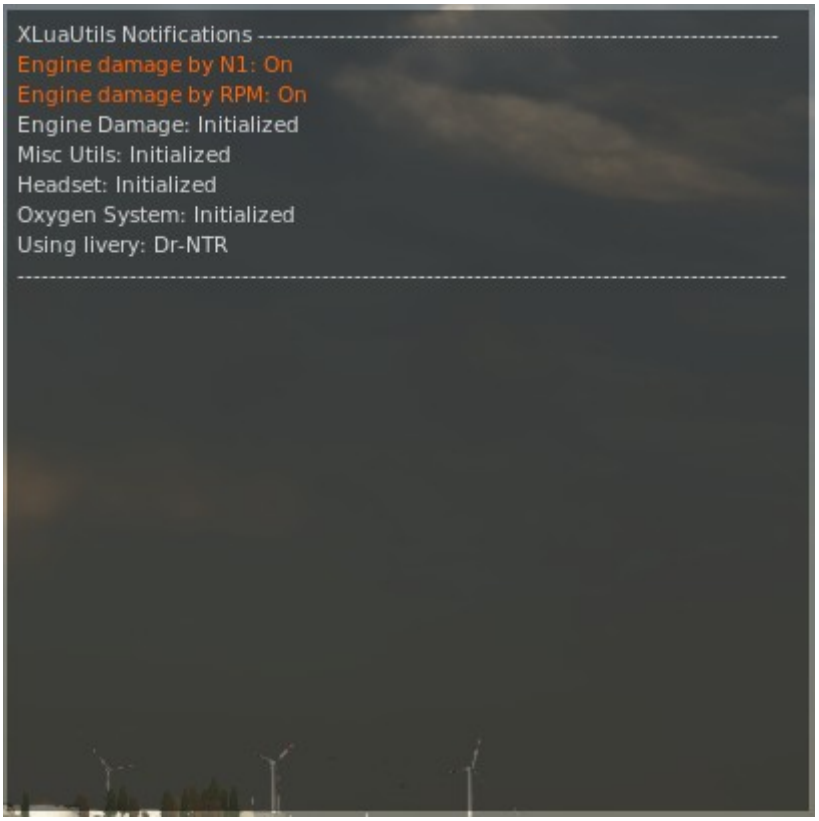


Image 12: Notifications in the XLuaUtils window

Notifications are handled by means of a message stack table. This table is refreshed regularly, and time-limited notifications are automatically purged from it. If the message stack table's length is zero, the notification window will close. Use the functions below to interact with the stack.

3.8.2 Functions

Display a notification in the notification window with `DisplayNotification(…)`.

DisplayNotification(inputstring,colorkey,displaytime)		
Parameter	**Value(s)**	**Description**

DisplayNotification(inputstring,colorkey,displaytime)		
inputstring	[Any string]	An input string, e.g. “Hello”.
colorkey	“Nominal”, “Success”, “Caution”, “Warning”	“Nominal” is white, “Success” is green, “Caution” is orange and “Warning” is red.
displaytime	[Any number]	Positive numbers define the amount of time in seconds that a notification will display, any negative number produces a pinned notification with the negative number serving as ID. Each pinned notification must have a unique, numerical ID!

Check the status of a pinned notification with *CheckNotification(…)*. Returns “true” if a notification was found in the stack by its ID.

CheckNotification(inID)		
Parameter	**Value(s)**	**Description**
inID	[Negative number]	The ID of the notification.

Remove a pinned notification with *RemoveNotification(...)*.

RemoveNotification(inID)		
Parameter	**Value(s)**	**Description**
inID	[Negative number]	The ID of the notification.

Update a pinned notification with *UpdateNotification(…)*.

UpdateNotification(inputstring,colorkey,inID)		
Parameter	**Value(s)**	**Description**
inputstring	[Any string]	An input string, e.g. “Hello”.
colorkey	“Nominal”, “Success”, “Caution”, “Warning”	“Nominal” is white, “Success” is green, “Caution” is orange and “Warning” is red.
inID	[Negative number]	The ID of the notification.

3.9 Dataref Handlers

XLuaUtils enables handling datarefs from an input table containing a list of datarefs. A table of input datarefs can be checked for validity and transferred into a container table which holds various information about each dataref. Said container table can then be used for further manipulation or storage in a save file.

References for dataref handling can be found in *XluaUtils/Submodules/xluautils_core_datarefs.lua, XluaUtils/Submodules/util_ncheadset.lua* and *xluautils/Submodules/util_misc.lua*.

3.9.1 Dataref Tables

To facilitate handling of datarefs, a simple **input table** is used in which each element consists of a sub-table with an alias string (used for easier lookup in the code) followed by the dataref stated as a string. If a specific alias for a dataref is not required, “Dref[n]” can be assigned to generate an alias from the table element’s index. See below for an example. All types of X-Plane datarefs are supported.
Note that, as of XLua 1.1 (or higher), these dataref tables have to be defined as local and are constrained to the XLuaUtils submodule in which they are declared.

```
local mydreflist={

{"Volume_Fan","sim/operation/sound/fan_volume_ratio"}, -- Actual X-Plane
dataref, will be found and stored for use later on

{"My_fake_dref","sim/fake/dataref"}, -- Will be discarded because it does not
exist

{"Dref[n]","sim/operation/sound/interior_volume_ratio"}, -- Receives "Dref3"
as an alias; actual X-Plane dataref, will be found and stored for use later
on

}
```

A dataref container table should be declared as follows, with the fist item being a unique identifier for storage in a save file:

```
local mydrefcontainer = {

"DATAREF",

}
```

Issuing `DrefTable_Read(mydreflist,mydrefcontainer)` in a script startup routine will check if the datarefs in *“mydreflist”* exist in X-Plane. Invalid datarefs (such as the fake one in the example or non-writable ones) will be discarded.
A successfully populated container table with the input dataref examples above has the following format:

```
local mydrefcontainer = {

"DATAREF",

-- alias,dataref,dataref type,{dataref value(s) storage 1},{dataref value(s)
storage 2},dataref handle

{"Volume_Fan","sim/operation/sound/fan_volume_ratio",2,{1},{},cdata<void *>:
0x08e409d0}

{"Dref3","sim/operation/sound/interior_volume_ratio",2,{1},{},cdata<void *>:
0x08e40480}

}
```

There are two subtables for storing dataref values at table index 4 and 5. This can be used to, for example, store default dataref values at X-Plane session start and restore them later on or for storing historic dataref values to determine a dataref delta over time.
The length of these subtables corresponds to the length of the dataref and supports array type datarefs.

3.9.2 Reading/Writing from/to Datarefs

The function `Dataref_Read(intable,subtable,filter)` reads the value of one or all datarefs in a container table from X-Plane. The parameter *"intable"* defines the dataref container table to be used, *"subtable"* defines the sub table that the dataref's value(s) is/are to be stored in (either index 4 or 5) and *"filter"* may either be the alias of a specific dataref in the container table or "All" to iterate over the entire container table and update each dataref from X-Plane.
Using the example table above, the function call to update the entire dataref table using storage position 1 at table element index 4 would be: `Dataref_Read(mydrefcontainer,4,"All")`

Manipulation of the dataref values stored inside the container table is not covered here.

Writing back values from the container table to X-Plane's corresponding datarefs is done with `Dataref_Write(intable,subtable,filter)`, where the parameter *"intable"* defines the dataref container table to be used, *"subtable"* defines the sub table that the dataref's value(s) is/are to be read from (either index 4 or 5) and *"filter"* may either be the alias of a specific dataref in the container table or "All" to iterate over the entire container table and write each dataref to X-Plane.
Again using the example container table above, writing to X-Plane from storage position 2 (table element index 5) and only updating *"Dref3"* (the alias of *"sim/operation/sound/interior_volume_ratio"*, see above) would be: `Dataref_Read(mydrefcontainer,5,"Dref3").`